AegisNet

Autonomous Multi-Agent SecOps Engine

Deep Engineering Specification & Architecture Blueprint

Executive Project Summary

AegisNet is a comprehensive, open-source active defense simulation network. Traditional cybersecurity relies on static signature matching, which fails against zero-day exploits and adversarial evasion techniques. AegisNet solves this by synthesizing multi-modal AI: it monitors tabular firewall logs (XGBoost), converts suspicious executable binary files into images for structural classification (Vision Transformers), queries cybersecurity compliance frameworks (Neo4j GraphRAG), and uses a Deep Reinforcement Learning (DRL) agent to isolate compromised hosts before lateral movement occurs.

1 1. System Architecture & Data Contracts

The system operates on a Directed Acyclic Graph (DAG) governed by a stateful LangGraph agent swarm. The architecture relies on strict JSON data contracts between microservices.

- **Layer 1 - Ingestion (The Senses):** Streaming netflow data is parsed via a FastAPI endpoint. An XGBoost model evaluates the payload.
- **Layer 2 - Vision Triage (The Analyst):** If anomaly probability $p > 0.85$, raw binary payloads are intercepted. Binaries are mapped from 1D byte arrays into 2D visual matrices. A PyTorch ViT categorizes the structural texture into threat families.
- **Layer 3 - Graph Retrieval (The Brain):** The classified threat identifier is sent to the LLM orchestration layer. The LLM generates Cypher queries to Neo4j to map the threat against the MITRE ATT&CK framework and enterprise topologies.
- **Layer 4 - Active Defense (The Muscle):** Intelligence from Layer 3 parameterizes a Deep Q-Network (DQN) in *CyberBattleSim*. The DRL agent executes node isolation, firewall updates, and decoy deployment.
- **Layer 5 - MLOps (The Overwatch):** Containerized via Docker. Evidently AI monitors incoming statistical distributions for data drift using Wasserstein distances.

2 2. Comprehensive Directory of Tools & Datasets

Domain	Technology Stack	Dataset / Data Source
Log ML	Pandas, Polars, XGBoost 2.0, Optuna, SMOTE.	**CICIDS2017**: 2.8M+ rows, 78+ network traffic features.
Vision ML	PyTorch 2.2, Torchvision, ONNX Runtime, ViT-Base.	**Malimg Dataset**: 9,339 malware images across 25 families.
GraphRAG	Neo4j, LangChain, SentenceTransformers (`all-MiniLM`).	**MITRE ATT&CK STIX** dataset, NVD JSON API feeds.
Orchestration	LangGraph, FastAPI, Pydantic (data validation).	Local LLM (e.g., Llama-3 8B) or OpenAI API.
Reinforcement	Stable-Baselines3 (PPO/DQN), Gymnasium API.	**Microsoft CyberBattleSim** (pip installable environment).
MLOps	Docker Compose, Evidently AI, Prometheus, Grafana.	Live inference telemetry captured in JSON logs.

3 3. Engineering Roadmap & Technical Specs

3.1 Phase 1: The Intrusion Filtering Backbone

Objective: Build a high-throughput tabular classifier to filter raw network streams.

- **Feature Engineering:** Load CICIDS2017 via Polars. **CRITICAL:** Drop 'Source IP', 'Destination IP', and 'Timestamp' features. If left in, the model will memorize specific IP addresses instead of learning the underlying attack behavior.
- **Optimization:** Use `Optuna` for Bayesian hyperparameter tuning. Target metrics: `max_depth` (3 to 9), `learning_rate` (0.01 to 0.1), and `subsample` (0.6 to 1.0).
- **Evaluation:** Due to massive class imbalance (90% benign traffic), optimize for **Macro F1-Score** rather than pure accuracy.

3.2 Phase 2: Automated Malware Imaging Pipeline

Objective: Identify polymorphic malware via visual texture without executing malicious code.

- **Byte-to-Image Algorithm:** Read raw `.bytes` file as 8-bit integers (0-255). Reshape into a 2D array. The matrix width (W) is dynamically determined by file size to maintain structural aspect ratios:

File Size	Image Width (W)	File Size	Image Width (W)
$<$ 10 kB	32	100 kB – 200 kB	384
10 kB – 30 kB	64	200 kB – 500 kB	512
30 kB – 60 kB	128	500 kB – 1000 kB	768
60 kB – 100 kB	256	$>$ 1000 kB	1024

- **Augmentation:** Apply `CutMix` and `MixUp` augmentations in PyTorch during training to force the Vision Transformer (ViT) to focus on global structural patterns rather than localized artifacts.

3.3 Phase 3: The MITRE ATT&CK Spatial GraphRAG

Objective: Provide the orchestration LLM with deterministic, structured context.

- **Graph Schema Definition:** Nodes: `(Subnet)`, `(Server)`, `(Vulnerability)`, `(MalwareFamily)`. Edges: `[:HOSTS]`, `[:EXPLOITS]`, `[:MITIGATES]`.
- **Vector Embedding Property:** Store `SentenceTransformer` embeddings of CVE descriptions directly in the `Vulnerability` nodes.
- **Query Execution:** When Phase 2 detects "Worm.Autorun", the LangGraph agent executes a Cypher query to determine lateral movement risk:

```
// Example GraphRAG Retrieval Query
MATCH (m:MalwareFamily {name: "Worm.Autorun"})-[:EXPLOITS]->(v:Vulnerability)
MATCH (v)<-[:AFFECTS]-(s:Server)<-[:HOSTS]-(sub:Subnet)
RETURN sub.ip_range, s.hostname, v.cve_id, v.mitigation_steps
ORDER BY v.cvss_score DESC LIMIT 5;
```

3.4 Phase 4: Active Defense Simulation & Agent Swarm

Objective: Automate the incident response protocol using DRL and LLM oversight.

- **DRL Reward Function (R_t):** The Stable-Baselines3 PPO agent operates under a strict penalty/reward mathematical framework to prevent self-induced denial of service:
  - $R = -100$: Agent isolates a critical benign node (False Positive).
  - $R = +50$: Agent successfully isolates a compromised node.
  - $R = -1$: Time penalty per step to encourage rapid threat containment.
- **LangGraph State Schema:** The LLM manages a global state dictionary bridging all phases:

```
class SecOpsState(TypedDict):
    network_alert: dict         # Phase 1: XGBoost output
    vision_classification: str  # Phase 2: ViT malware lineage
    graph_context: list         # Phase 3: Neo4j mitigations
    proposed_drl_action: str    # Phase 4: CyberBattleSim action
    human_in_loop_approval: bool
```

3.5 Phase 5: Production MLOps & Tracking Layer

Objective: Ensure real-time performance and prevent silent model degradation.

- **Inference Acceleration:** Export the PyTorch ViT and XGBoost models to `ONNX` graph format. This bypasses the Python Global Interpreter Lock (GIL) and reduces inference latency from $\sim$120ms to $<$15ms per payload.
- **Evidently AI Drift Detection:** Continuous features (like `Packet_Length_Mean`) are monitored using the **Wasserstein Distance** metric. Categorical features use **Jensen-Shannon divergence**. If drift exceeds 0.15, Prometheus triggers an automated Slack/Teams alert for model retraining.

- **Deployment Topology:** Deployed via `docker-compose.yml` spinning up 4 containers: 1) FastAPI Inference Server, 2) Neo4j Database, 3) Evidently AI Metrics Dashboard, 4) LLM Orchestration Worker.

4 4. System Testing & Real-World Implementation

4.1 Testing & Validation Protocols

To prove AegisNet works prior to deployment, the system must undergo rigorous adversarial validation.

- **Traffic Replay (Unit Testing):** Use `tcpreplay` to inject raw PCAP (Packet Capture) files from the CICIDS2017 dataset into your local network interface. This validates that Layer 1 (XGBoost) can process live streaming packets in real-time, not just static CSV files.
- **Adversarial Simulation (Integration Testing):** Deploy **MITRE Caldera** (an open-source automated adversary emulation system) within your CyberBattleSim environment. Command Caldera to execute a specific advanced persistent threat profile (e.g., APT29) to see if your LangGraph swarm successfully detects and halts the lateral movement.
- **State Machine Auditing:** Write `PyTest` scripts to mock the LangGraph state. Assert that if the LLM receives a "High Confidence Trojan" flag from the Vision Triage layer, it *always* successfully issues the "Isolate Node" command to the Reinforcement Learning agent.

4.2 Real-World Enterprise Integration

While AegisNet is trained in a simulator, its architecture is designed for plug-and-play integration with live enterprise security stacks.

Production Deployment Topology

Step 1: **Data Ingestion (The Sensor):** AegisNet does not sit inline (which could slow down legitimate traffic). Instead, it connects to a **SPAN/Mirror Port** on the core network switch. Open-source sensors like **Zeek** or **Suricata** convert the raw packet bytes into structured JSON logs and forward them to the Phase 1 FastAPI endpoint.

Step 2: **File Extraction:** If Zeek detects an executable file transfer over HTTP/SMB, it automatically extracts the `.exe`/`.bin` payload and forwards it directly to the Phase 2 Vision Triage endpoint via an API `POST` request.

Step 3: **SIEM Integration:** Instead of just printing alerts to a terminal, AegisNet pushes its GraphRAG intelligence and threat scores to a Security Information and Event Management (SIEM) platform like **Elastic Security (ELK Stack)** or **Splunk** via standard syslog or JSON webhooks.

Step 4: **Active Response Execution (SOAR):** When the Deep RL agent decides to isolate a compromised node, AegisNet does not hack the node itself. It acts as a Security Orchestration, Automation, and Response (SOAR) tool. It sends an authenticated API payload to a network firewall (e.g., open-source **pfSense** or enterprise **Palo Alto**) or an EDR agent (e.g., **CrowdStrike**) to dynamically update the Access Control List (ACL) and quarantine the infected IP address.

5 5. Hardware & System Requirements

Because AegisNet utilizes multi-modal models alongside graph databases, it requires specific computational resources. The entire stack is designed to run locally to preserve data privacy.

- **Minimum Prototyping Specs (CPU Only):**
  - **RAM:** 16 GB DDR4 (Required for Neo4j + Polars dataframes).
  - **CPU:** 8-Core processor (e.g., Intel i7 or AMD Ryzen 7).
  - **Storage:** 50 GB SSD (For CICIDS2017 logs and Malimg image datasets).
  - *Note:* In minimum spec, the LLM must be accessed via external API (e.g., OpenAI / Groq) rather than hosted locally.
- **Recommended Production Specs (Local LLM & Accelerated CV):**
  - **RAM:** 32 GB+ DDR4/DDR5.
  - **GPU:** NVIDIA RTX 3060 / 4070 (12GB+ VRAM) or Apple Silicon (M2/M3 Max). Required to run the Llama-3 8B orchestration agent locally via Ollama, and for < 20ms Vision Transformer inference.
  - **OS:** Ubuntu 22.04 LTS or WSL2 on Windows 11 (Docker optimized).
- **Cloud-Accelerated Model Training (Alternative):**
  - For the computationally heavy initial training phases (specifically training the Vision Transformer in Phase 2 and running Optuna hyperparameter tuning in Phase 1), it is highly recommended to use **Google Colab** or **Kaggle Notebooks**.
  - Both platforms provide free access to cloud GPUs (e.g., NVIDIA T4/P100) and high-RAM environments, which will drastically reduce the time needed to process the 2.8M+ rows of the CICIDS2017 dataset and the Malimg image tensors. Once trained, the resulting `.onnx` or `.pt` weights can be downloaded to your local machine for the active defense inference pipeline.

6 6. Safety Guardrails & Human-in-the-Loop (HITL)

Fully autonomous SecOps presents a risk of self-inflicted Denial of Service (DoS) if the agent blocks legitimate business traffic (False Positives). AegisNet mitigates this via a deterministic fallback protocol within the LangGraph State Machine.

The HITL Approval Matrix

The LangGraph `Execution Node` calculates an aggregate *Confidence Score* (C) based on Phase 1 (XGBoost probability) and Phase 2 (ViT softmax confidence).

1. **Condition Green ($C > 0.95$ & Target is User Endpoint):** The DRL agent is granted **Full Autonomy**. The firewall rule is executed instantly without human intervention.
2. **Condition Yellow ($0.75 < C \leq 0.95$):** The DRL agent halts. The LangGraph agent packages the GraphRAG summary and sends an interactive webhook to a designated Slack/Microsoft Teams channel. A human analyst must click `[Approve]` or `[Reject]` to proceed.
3. **Condition Red (Target is a Core Infrastructure Node):** Regardless of the confidence score, if the node targeted for isolation is tagged in Neo4j as `Role: Domain_Controller` or `Role: Core_Database`, the autonomous action is **strictly blocked**. The agent may

only deploy passive honeypots (decoys) while paging the human incident response team.